Experiment 1: Feature Transformation - LDA on Iris Dataset

Source Code

```
# Experiment 1: Feature Transformation - LDA on the Iris dataset
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import accuracy_score, confusion_matrix,
    classification_report

# 1. Load data: Kaggle "Iris.csv" if it is in this folder, else scikit-
    learn's copy
if os.path.exists("Iris.csv"):
    df = pd.read_csv("Iris.csv").drop(columns=["Id"], errors="ignore")
    X, y = df.iloc[:, :4], df["Species"]
    print("Data source: Iris.csv")
else:
    iris = load_iris(as_frame=True)
    X = iris.data
    y = pd.Series(iris.target_names[iris.target], name="Species")
    print("Data source: scikit-learn built-in Iris (same 150 samples)")

# 2. Inspect the data
print("\nShape:", X.shape)
print("\nFirst 5 rows:\n", X.head())
print("\nMissing values per column:\n", X.isnull().sum())
print("\nClass counts:\n", y.value_counts())

# 3. Train / test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

# 4. Fit LDA (also used as a classifier)
lda = LinearDiscriminantAnalysis(n_components=2)
lda.fit(X_train, y_train)

# 5. Dimensionality reduction: 4 features -> 2 discriminants
X_train_lda = lda.transform(X_train)
X_test_lda = lda.transform(X_test)
print("\nReduced shape:", X_train.shape, "->", X_train_lda.shape)
print("Explained variance ratio:", np.round(lda.explained_variance_ratio_,
    4))

# 6. Predict and evaluate
y_pred = lda.predict(X_test)
print("\nAccuracy: {:.2f}%".format(accuracy_score(y_test, y_pred) * 100))
print("\nConfusion matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification report:\n", classification_report(y_test, y_pred))
# 7. Classify a new flower
```

```
sample = pd.DataFrame([[5.1, 3.5, 1.4, 0.2]], columns=X.columns)
print("Sample flower:", sample.values.tolist()[0])
print("Predicted species:", lda.predict(sample)[0])

# 8. Plot the LDA projection
plt.figure(figsize=(7, 5))
for sp in np.unique(y_train):
    m = (y_train == sp).values
    plt.scatter(X_train_lda[m, 0], X_train_lda[m, 1], label=sp)
plt.xlabel("LD1"); plt.ylabel("LD2")
plt.title("Iris data projected onto LDA components")
plt.legend(); plt.grid(alpha=0.3)
plt.show()
```

Experiment 1: Feature Transformation - LDA on Iris Dataset

Output

```
Data source: scikit-learn built-in Iris (same 150 samples)

Shape: (150, 4)

First 5 rows:
    sepal length (cm)  sepal width (cm)  petal length (cm)  petal width
        (cm)
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

Missing values per column:
 sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
dtype: int64

Class counts:
 Species
setosa        50
versicolor    50
virginica     50
Name: count, dtype: int64

Reduced shape: (105, 4) -> (105, 2)
Explained variance ratio: [0.9882 0.0118]

Accuracy: 97.78%

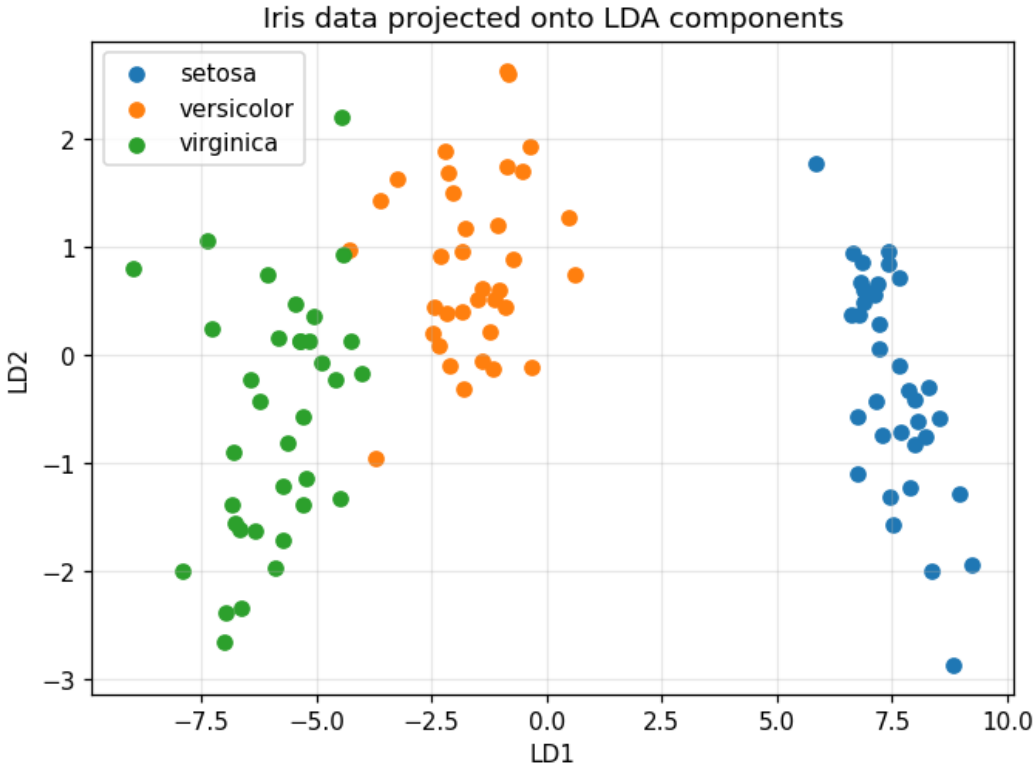
Confusion matrix:
 [[15  0  0]
 [ 0 15  0]
 [ 0  1 14]]

Classification report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        15
  versicolor       0.94      1.00      0.97        15
   virginica       1.00      0.93      0.97        15

    accuracy                           0.98        45
   macro avg       0.98      0.98      0.98        45
weighted avg       0.98      0.98      0.98        45

Sample flower: [5.1, 3.5, 1.4, 0.2]
Predicted species: setosa
```


Iris data projected onto LDA components
setosa
versicolor
virginica
LD1
LD2
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
2
1
0
−1
−2
−3

Experiment 2: Regression Analysis - Uber Fare Prediction (Linear, Ridge, Lasso)

Source Code

```
# Experiment 2: Uber fare prediction - Linear, Ridge and Lasso regression
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score, mean_squared_error,
    mean_absolute_error

# 1. Load data: Kaggle "uber.csv" if present, else a synthetic dataset with
     the same columns
if os.path.exists("uber.csv"):
    df = pd.read_csv("uber.csv")
    print("Data source: uber.csv")
else:
    rng = np.random.default_rng(42)
    n = 20000
    lat1 = rng.uniform(40.62, 40.88, n); lon1 = rng.uniform(-74.03, -73.78,
         n)
    lat2 = lat1 + rng.normal(0, 0.04, n); lon2 = lon1 + rng.normal(0, 0.04,
         n)
    km = np.sqrt(((lat2 - lat1) * 111) ** 2 + ((lon2 - lon1) * 84) ** 2)
    t = pd.to_datetime("2012-01-01") + pd.to_timedelta(rng.integers(0, 3 *
        365 * 24 * 60, n), unit="m")
    fare = 2.5 + 1.8 * km + 0.4 * (t.year - 2012) + rng.normal(0, 1.2, n)
    df = pd.DataFrame({"fare_amount": fare.round(2), "pickup_datetime":
        t.astype(str) + " UTC",
                       "pickup_longitude": lon1, "pickup_latitude": lat1,
                       "dropoff_longitude": lon2, "dropoff_latitude": lat2,
                       "passenger_count": rng.integers(1, 7, n)})
    bad = rng.choice(n, 150, replace=False)               # inject dirty
        rows / outliers
    df.loc[bad[:50], "fare_amount"] = rng.uniform(150, 500, 50)
    df.loc[bad[50:80], "fare_amount"] = -rng.uniform(1, 5, 30)
    df.loc[bad[80:110], ["pickup_latitude", "pickup_longitude"]] = 0
    df.loc[bad[110:130], "passenger_count"] = 0
    df.loc[bad[130:140], "dropoff_latitude"] = np.nan
    print("Data source: SYNTHETIC fallback (place uber.csv next to this
        file to use the real data)")

# 2. Pre-process
df = df.drop(columns=[c for c in ["Unnamed: 0", "key"] if c in df.columns])
print("\nShape:", df.shape)
print("Missing values:\n", df.isnull().sum()[df.isnull().sum() > 0])
df = df.dropna()
df["pickup_datetime"] = pd.to_datetime(df["pickup_datetime"], utc=True)
df["hour"] = df["pickup_datetime"].dt.hour
```

```
df["weekday"] = df["pickup_datetime"].dt.weekday
df["month"] = df["pickup_datetime"].dt.month
df["year"] = df["pickup_datetime"].dt.year

def haversine(lat1, lon1, lat2, lon2):               # distance in km
    p = np.pi / 180
    a = (np.sin((lat2 - lat1) * p / 2) ** 2
         + np.cos(lat1 * p) * np.cos(lat2 * p) * np.sin((lon2 - lon1) * p /
             2) ** 2)
    return 12742 * np.arcsin(np.sqrt(a))

df["distance_km"] = haversine(df.pickup_latitude, df.pickup_longitude,
                              df.dropoff_latitude, df.dropoff_longitude)
before = len(df)
df = df[(df.fare_amount > 0) & df.passenger_count.between(1, 6)
        & df.pickup_latitude.between(40, 42) &
            df.dropoff_latitude.between(40, 42)
        & df.pickup_longitude.between(-75, -72) &
            df.dropoff_longitude.between(-75, -72)]
print("Invalid rows removed:", before - len(df))

# 3. Identify outliers (IQR rule) + box plots
def iqr_bounds(s):
    q1, q3 = s.quantile([0.25, 0.75]); i = q3 - q1
    return q1 - 1.5 * i, q3 + 1.5 * i

fig, ax = plt.subplots(1, 2, figsize=(9, 4))
df.boxplot(column="fare_amount", ax=ax[0]); ax[0].set_title("fare_amount")
df.boxplot(column="distance_km", ax=ax[1]); ax[1].set_title("distance_km")
plt.tight_layout(); plt.show()
for col in ["fare_amount", "distance_km"]:
    lo, hi = iqr_bounds(df[col])
    print(f"{col}: bounds ({lo:.2f}, {hi:.2f}) -> {((df[col] < lo) |
        (df[col] > hi)).sum()} outliers")
lo, hi = iqr_bounds(df.fare_amount); df = df[df.fare_amount.between(lo,
    hi)]
lo, hi = iqr_bounds(df.distance_km); df = df[df.distance_km.between(lo,
    hi)]
print("Shape after outlier removal:", df.shape)

# 4. Correlation
num = df[["fare_amount", "distance_km", "passenger_count", "hour",
    "weekday", "month", "year"]]
corr = num.corr()
print("\nCorrelation with fare_amount:\n", corr["fare_amount"].round(3))
plt.figure(figsize=(7, 5.5))
plt.imshow(corr, cmap="coolwarm", vmin=-1, vmax=1); plt.colorbar()
plt.xticks(range(len(corr)), corr.columns, rotation=45, ha="right")
plt.yticks(range(len(corr)), corr.columns)
for i in range(len(corr)):
    for j in range(len(corr)):
        plt.text(j, i, f"{corr.iloc[i, j]:.2f}", ha="center", va="center",
            fontsize=8)
plt.title("Correlation matrix"); plt.tight_layout(); plt.show()
```

```
# 5. Train / test split and scaling
features = ["distance_km", "passenger_count", "hour", "weekday", "month",
    "year"]
X, y = df[features], df["fare_amount"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)
sc = StandardScaler()
X_train_s, X_test_s = sc.fit_transform(X_train), sc.transform(X_test)

# 6. Linear, Ridge and Lasso regression
models = {"Linear": LinearRegression(), "Ridge": Ridge(alpha=1.0), "Lasso":
     Lasso(alpha=0.01)}
rows = []
for name, m in models.items():
    m.fit(X_train_s, y_train)
    p = m.predict(X_test_s)
    rows.append([name, r2_score(y_test, p),
        np.sqrt(mean_squared_error(y_test, p)),
                 mean_absolute_error(y_test, p)])

# 7. Compare
res = pd.DataFrame(rows, columns=["Model", "R2", "RMSE",
    "MAE"]).set_index("Model")
print("\nModel comparison:\n", res.round(4))
print("\nLasso coefficients (0 = feature dropped):")
print(pd.Series(models["Lasso"].coef_, index=features).round(3))
res[["R2"]].plot(kind="bar", legend=False, ylim=(res.R2.min() - 0.01, 1),
    rot=0)
plt.title("R2 score by model"); plt.ylabel("R2"); plt.show()
```

Experiment 2: Regression Analysis - Uber Fare Prediction (Linear, Ridge, Lasso)

Output

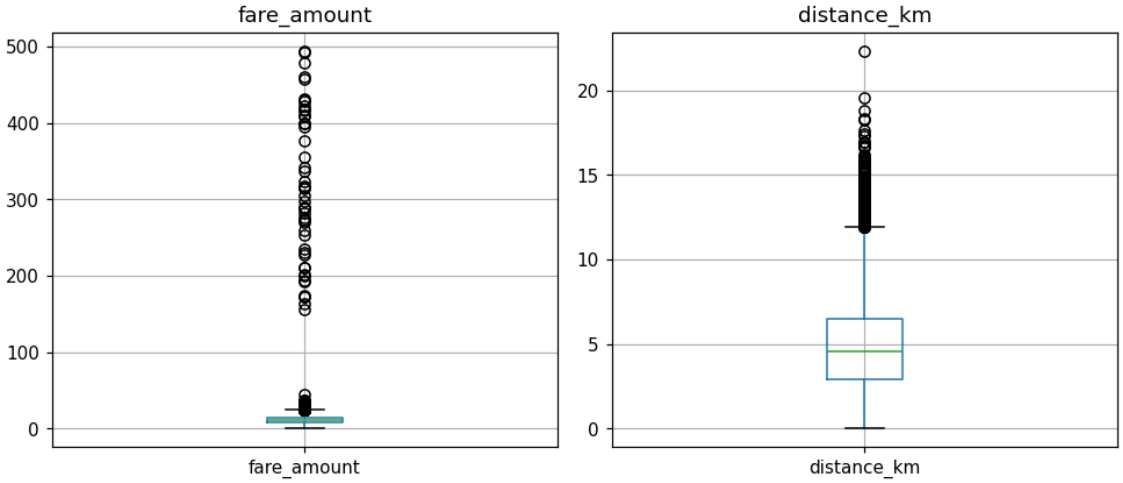
```
Data source: SYNTHETIC fallback (place uber.csv next to this file to use
    the real data)

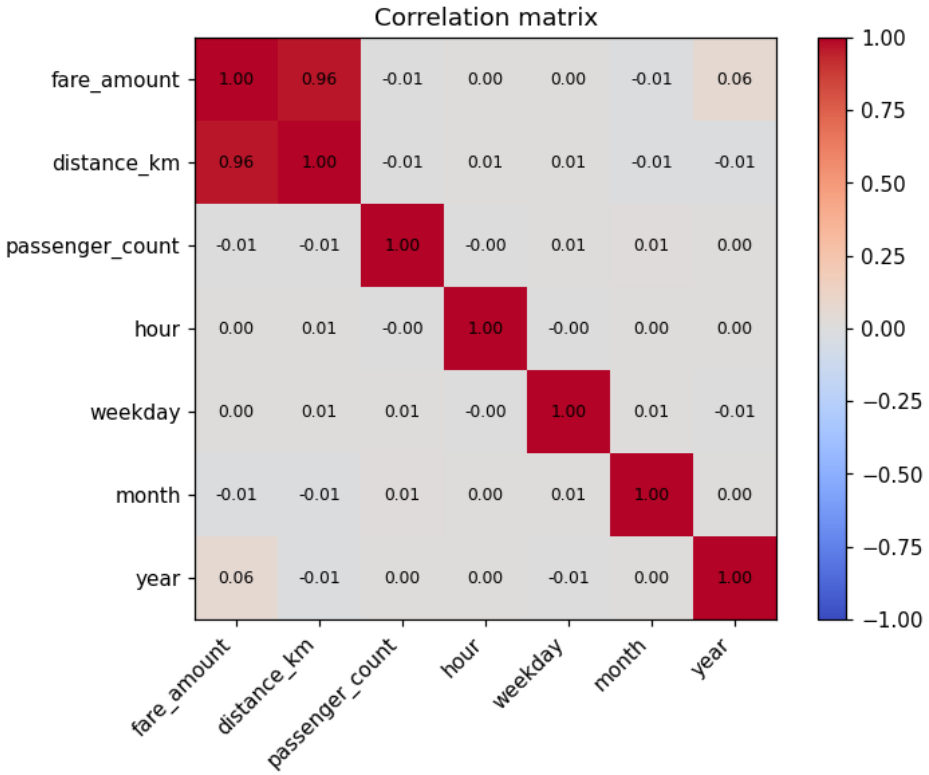
Shape: (20000, 7)
Missing values:
 dropoff_latitude    10
dtype: int64
Invalid rows removed: 81
fare_amount: bounds (-1.92, 24.75) -> 296 outliers
distance_km: bounds (-2.47, 11.91) -> 271 outliers
Shape after outlier removal: (19536, 12)

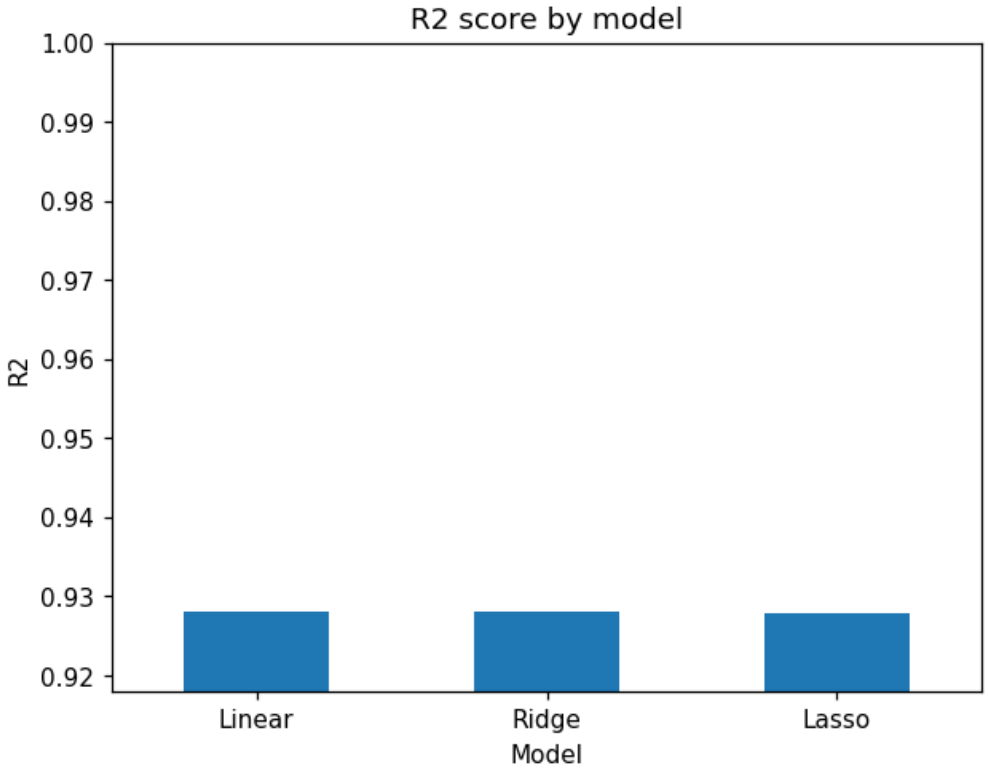
Correlation with fare_amount:
 fare_amount        1.000
distance_km         0.962
passenger_count    -0.006
hour                0.004
weekday             0.005
month              -0.008
year                0.061
Name: fare_amount, dtype: float64

Model comparison:
              R2    RMSE     MAE
Model
Linear  0.9280  1.2028  0.9603
Ridge   0.9280  1.2028  0.9603
Lasso   0.9279  1.2031  0.9606

Lasso coefficients (0 = feature dropped):
distance_km         4.382
passenger_count    -0.000
hour               -0.000
weekday            -0.000
month               0.000
year                0.313
dtype: float64
```


fare_amount
distance_km
500
400
300
200
100
0
20
15
10
5
0
fare_amount
distance_km


Correlation matrix
fare_amount
distance_km
passenger_count
hour
weekday
month
year
1.00 0.96 -0.01 0.00 0.00 -0.01 0.06
0.96 1.00 -0.01 0.01 0.01 -0.01 -0.01
-0.01 -0.01 1.00 -0.00 0.01 0.01 0.00
0.00 0.01 -0.00 1.00 -0.00 0.00 0.00
0.00 0.01 0.01 -0.00 1.00 0.01 -0.01
-0.01 -0.01 0.01 0.00 0.01 1.00 0.00
0.06 -0.01 0.00 0.00 -0.01 0.00 1.00
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00


R2 score by model
1.00
0.99
0.98
0.97
0.96
0.95
0.94
0.93
0.92
R2
Linear
Ridge
Lasso
Model

Experiment 3: Classification - SVM for Handwritten Digits

Source Code

```
# Experiment 3: SVM for handwritten digit classification (0-9)
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn import metrics

# 1. Load data: Kaggle Digit Recognizer "train.csv" (label + 784 pixels) if
     present,
#    else scikit-learn's built-in 8x8 digits (label + 64 pixels)
if os.path.exists("train.csv"):
    train_df = pd.read_csv("train.csv", nrows=10000)   # remove nrows to
        use all 42000 rows
    X = train_df.drop("label", axis=1)
    y = train_df["label"]
    print("Data source: train.csv")
else:
    d = load_digits(as_frame=True)
    X, y = d.data, d.target
    print("Data source: scikit-learn built-in digits (8x8 images)")
side = int(np.sqrt(X.shape[1]))                          # 28 for MNIST, 8
    for built-in
print("Samples:", X.shape[0], "| Features:", X.shape[1], f"(image
    {side}x{side})")
print("Class counts:\n", y.value_counts().sort_index().to_dict())

# 2. Train / test split (30 % test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    random_state=100)

# 3. Standardise features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 4. Linear SVM (for comparison) and RBF SVM
lin_model = SVC(kernel="linear").fit(X_train, y_train)
acc_lin = metrics.accuracy_score(y_test, lin_model.predict(X_test))
rbf_model = SVC(kernel="rbf").fit(X_train, y_train)
y_rbf_pred = rbf_model.predict(X_test)

# 5. Compare predictions with actual labels
print("\nPredicted values:\n", y_rbf_pred[10:15])
print("Actual values:\n", y_test[10:15].values)

# 6. Accuracy
```

```
acc_rbf = metrics.accuracy_score(y_test, y_rbf_pred)
print("\nLinear kernel accuracy: {:.2f} %".format(acc_lin * 100))
print("RBF kernel accuracy:    {:.2f} %".format(acc_rbf * 100))
print("\nClassification report (RBF):\n",
    metrics.classification_report(y_test, y_rbf_pred))

# 7. Show a few test digits with the predicted label
np.random.seed(1)
fig, axes = plt.subplots(1, 4, figsize=(10, 3))
for ax, i in zip(axes, np.random.randint(0, len(X_test), 4)):
    img = scaler.inverse_transform(X_test[i:i + 1]).reshape(side, side)
    ax.imshow(img, cmap="gray")
    ax.set_title(f"predicted label: {y_rbf_pred[i]}")
plt.tight_layout(); plt.show()
```

Experiment 3: Classification - SVM for Handwritten Digits

Output

```
Data source: scikit-learn built-in digits (8x8 images)
Samples: 1797 | Features: 64 (image 8x8)
Class counts:
 {0: 178, 1: 182, 2: 177, 3: 183, 4: 181, 5: 182, 6: 181, 7: 179, 8: 174,
     9: 180}

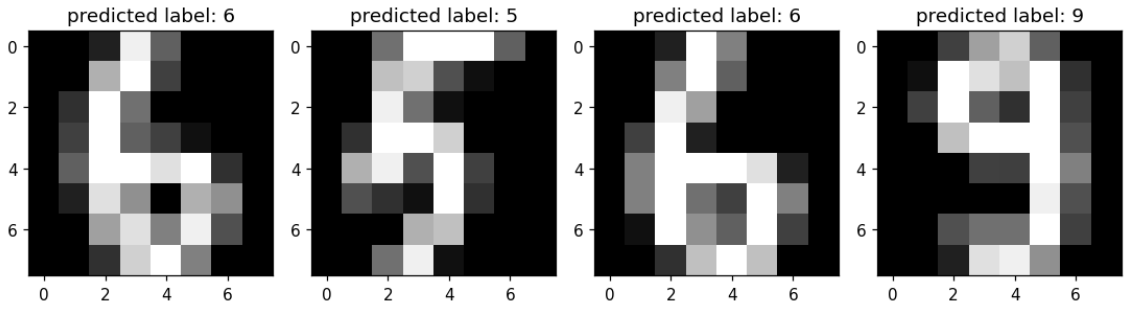
Predicted values:
 [4 5 7 5 9]
Actual values:
 [4 5 7 5 9]

Linear kernel accuracy: 98.52 %
RBF kernel accuracy:    98.89 %

Classification report (RBF):
               precision    recall  f1-score   support

            0       1.00      1.00      1.00        59
            1       1.00      0.98      0.99        50
            2       0.98      1.00      0.99        60
            3       1.00      1.00      1.00        49
            4       0.98      0.98      0.98        59
            5       1.00      0.98      0.99        54
            6       0.97      1.00      0.98        58
            7       1.00      1.00      1.00        50
            8       1.00      0.96      0.98        48
            9       0.96      0.98      0.97        53

    accuracy                           0.99       540
   macro avg       0.99      0.99      0.99       540
weighted avg       0.99      0.99      0.99       540
```



Experiment 4: Clustering - K-Means with Elbow Method on Iris

Source Code

```
# Experiment 4: K-Means clustering on Iris with the elbow method
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans

# 1. Load data (Iris.csv if present, else built-in copy)
if os.path.exists("Iris.csv"):
    df = pd.read_csv("Iris.csv").drop(columns=["Id"], errors="ignore")
    X = df.iloc[:, :4].values
    species = df["Species"]
    print("Data source: Iris.csv")
else:
    iris = load_iris(as_frame=True)
    X = iris.data.values
    species = pd.Series(iris.target_names[iris.target], name="Species")
    print("Data source: scikit-learn built-in Iris (same 150 samples)")
print("Shape:", X.shape)

# 2. Elbow method: WCSS for K = 1..10
wcss = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, init="k-means++", n_init=10,
        random_state=42).fit(X)
    wcss.append(km.inertia_)
    print(f"K = {k:2d}  WCSS = {km.inertia_:8.2f}")

plt.figure(figsize=(7, 4.5))
plt.plot(range(1, 11), wcss, marker="o")
plt.annotate("Elbow point (K = 3)", xy=(3, wcss[2]), xytext=(4.5, wcss[2] +
     120),
             arrowprops=dict(arrowstyle="->"))
plt.xlabel("Number of clusters"); plt.ylabel("WCSS"); plt.title("Elbow
    method")
plt.grid(alpha=0.3); plt.show()

# 3. Fit K-Means with the chosen K (the bend of the curve is at K = 3)
K = 3
kmeans = KMeans(n_clusters=K, init="k-means++", n_init=10, random_state=42)
labels = kmeans.fit_predict(X)
print("\nChosen K:", K)
print("Cluster sizes:", np.bincount(labels))
print("Centroids (sepal len, sepal wid, petal len, petal wid):\n",
    kmeans.cluster_centers_.round(3))
print("\nClusters vs actual species:\n", pd.crosstab(species, labels,
    rownames=["Species"], colnames=["Cluster"]))

# 4. Visualise the clusters (petal length vs petal width)
```

```
plt.figure(figsize=(7, 5))
for c in range(K):
    plt.scatter(X[labels == c, 2], X[labels == c, 3], label=f"Cluster {c}")
plt.scatter(kmeans.cluster_centers_[:, 2], kmeans.cluster_centers_[:, 3],
            s=200, c="black", marker="X", label="Centroids")
plt.xlabel("Petal length (cm)"); plt.ylabel("Petal width (cm)")
plt.title("K-Means clusters (K = 3)"); plt.legend(); plt.grid(alpha=0.3);
    plt.show()
```

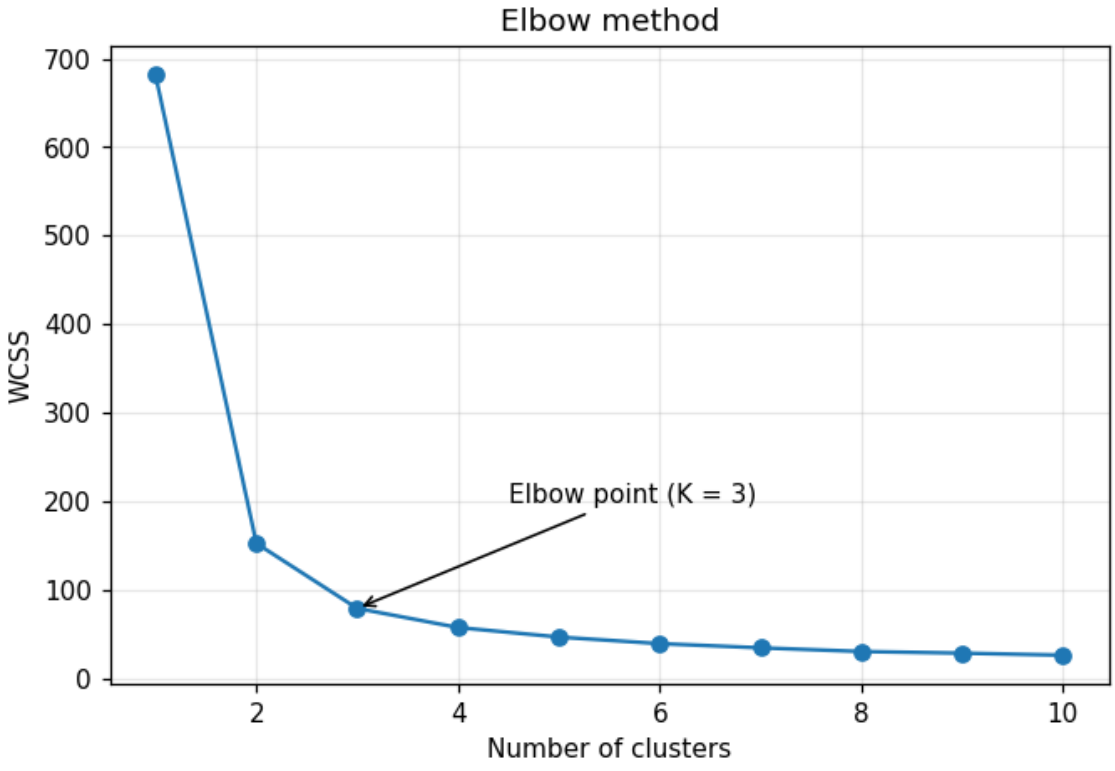
Experiment 4: Clustering - K-Means with Elbow Method on Iris

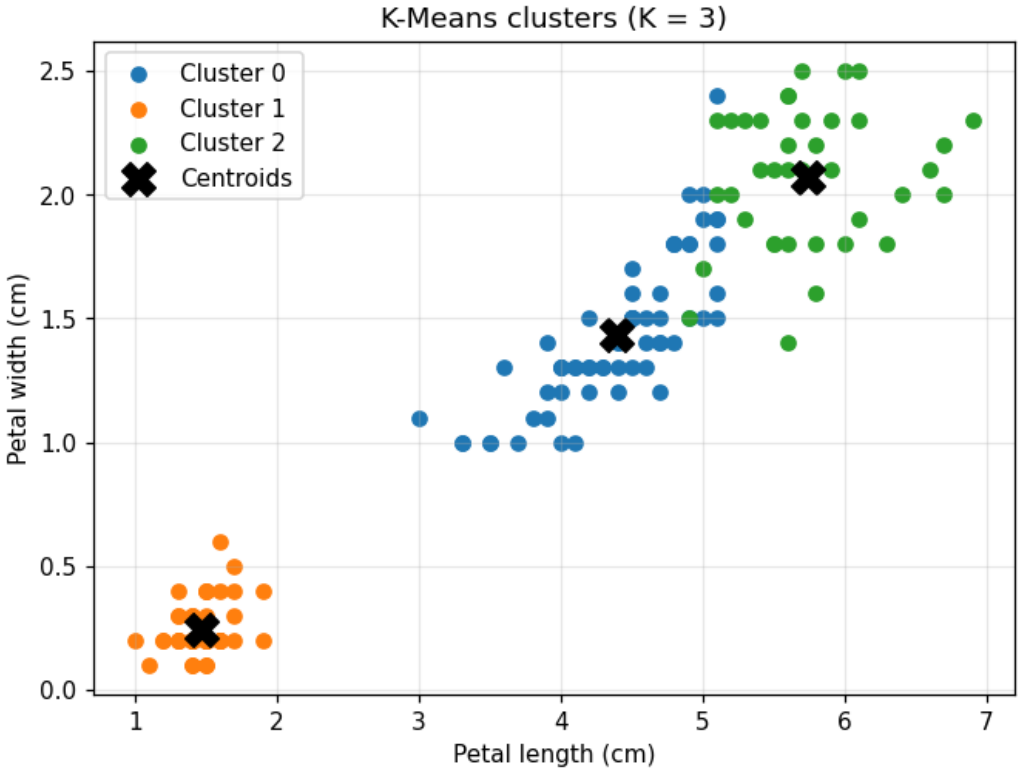
Output

```
Data source: scikit-learn built-in Iris (same 150 samples)
Shape: (150, 4)
K =  1  WCSS =   681.37
K =  2  WCSS =   152.35
K =  3  WCSS =    78.85
K =  4  WCSS =    57.23
K =  5  WCSS =    46.46
K =  6  WCSS =    39.04
K =  7  WCSS =    34.31
K =  8  WCSS =    30.13
K =  9  WCSS =    28.29
K = 10  WCSS =    25.97

Chosen K: 3
Cluster sizes: [62 50 38]
Centroids (sepal len, sepal wid, petal len, petal wid):
 [[5.902 2.748 4.394 1.434]
 [5.006 3.428 1.462 0.246]
 [6.85  3.074 5.742 2.071]]

Clusters vs actual species:
 Cluster      0   1   2
Species
setosa       0  50   0
versicolor  48   0   2
virginica   14   0  36
```




K-Means clusters (K = 3)
Cluster 0
Cluster 1
Cluster 2
Centroids
Petal width (cm)
Petal length (cm)
0.0
0.5
1.0
1.5
2.0
2.5
1
2
3
4
5
6
7

Experiment 5: Ensemble Learning - Random Forest for Car Safety

Source Code

```
# Experiment 5: Random Forest classifier - predict the safety class of a
    car
import os
import itertools
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OrdinalEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
    classification_report

cols = ["buying", "maint", "doors", "persons", "lug_boot", "safety",
    "class"]

# 1. Load data: Kaggle "car_evaluation.csv" (no header row) if present,
#    else a synthetic table covering all 1728 attribute combinations
if os.path.exists("car_evaluation.csv"):
    df = pd.read_csv("car_evaluation.csv", header=None, names=cols)
    print("Data source: car_evaluation.csv")
else:
    lv = {"buying": ["vhigh", "high", "med", "low"], "maint": ["vhigh",
        "high", "med", "low"],
          "doors": ["2", "3", "4", "5more"], "persons": ["2", "4", "more"],
          "lug_boot": ["small", "med", "big"], "safety": ["low", "med",
              "high"]}
    rows = []
    for b, m, d, p, l, s in itertools.product(*lv.values()):
        price = lv["buying"].index(b) + lv["maint"].index(m)          #
            0..6, higher = cheaper
        comfort = lv["doors"].index(d) + lv["persons"].index(p) +
            lv["lug_boot"].index(l)
        if s == "low" or p == "2" or price < 2:
            c = "unacc"
        elif price >= 5 and s == "high" and comfort >= 4:
            c = "vgood"
        elif price >= 4 and comfort >= 3:
            c = "good"
        else:
            c = "acc"
        rows.append([b, m, d, p, l, s, c])
    df = pd.DataFrame(rows, columns=cols)
    print("Data source: SYNTHETIC fallback (place car_evaluation.csv next
        to this file to use the real data)")

# 2. Inspect
print("\nShape:", df.shape)
print(df.head())
print("\nMissing values:", int(df.isnull().sum().sum()))
```

```
print("\nClass distribution:\n", df["class"].value_counts())

# 3. Encode categorical attributes
X = df.drop("class", axis=1)
y = df["class"]
order = [["low", "med", "high", "vhigh"], ["low", "med", "high", "vhigh"],
       # buying, maint
         ["2", "3", "4", "5more"], ["2", "4", "more"],
                                     # doors, persons
         ["small", "med", "big"], ["low", "med", "high"]]
                                    # lug_boot, safety
enc = OrdinalEncoder(categories=order)
X_enc = pd.DataFrame(enc.fit_transform(X), columns=X.columns)

# 4. Split
X_train, X_test, y_train, y_test = train_test_split(
    X_enc, y, test_size=0.3, random_state=42, stratify=y)

# 5. Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)

# 6. Evaluate
print("\nAccuracy: {:.2f} %".format(accuracy_score(y_test, y_pred) * 100))
print("\nConfusion matrix (rows = actual, cols = predicted):")
labels = sorted(y.unique())
print(pd.DataFrame(confusion_matrix(y_test, y_pred, labels=labels),
    index=labels, columns=labels))
print("\nClassification report:\n", classification_report(y_test, y_pred,
    zero_division=0))

# 7. Feature importance
imp = pd.Series(rf.feature_importances_, index=X.columns).sort_values()
print("Feature importance:\n", imp.sort_values(ascending=False).round(4))
imp.plot(kind="barh", figsize=(7, 4)); plt.title("Random Forest feature
    importance")
plt.xlabel("Importance"); plt.tight_layout(); plt.show()

# 8. Predict the class of a new car
new_car = pd.DataFrame([["low", "low", "4", "4", "big", "high"]],
    columns=X.columns)
print("\nNew car:", new_car.values.tolist()[0])
print("Predicted class:", rf.predict(pd.DataFrame(enc.transform(new_car),
    columns=X.columns))[0])
```

Experiment 5: Ensemble Learning - Random Forest for Car Safety

Output

```
Data source: SYNTHETIC fallback (place car_evaluation.csv next to this file
     to use the real data)

Shape: (1728, 7)
  buying  maint doors persons lug_boot safety  class
0  vhigh  vhigh     2       2    small    low  unacc
1  vhigh  vhigh     2       2    small    med  unacc
2  vhigh  vhigh     2       2    small   high  unacc
3  vhigh  vhigh     2       2      med    low  unacc
4  vhigh  vhigh     2       2      med    med  unacc

Missing values: 0

Class distribution:
 class
unacc    1104
acc       384
good      195
vgood      45
Name: count, dtype: int64

Accuracy: 99.04 %

Confusion matrix (rows = actual, cols = predicted):
       acc  good  unacc  vgood
acc    114     1      0      0
good     3    55      0      1
unacc    0     0    332      0
vgood    0     0      0     13

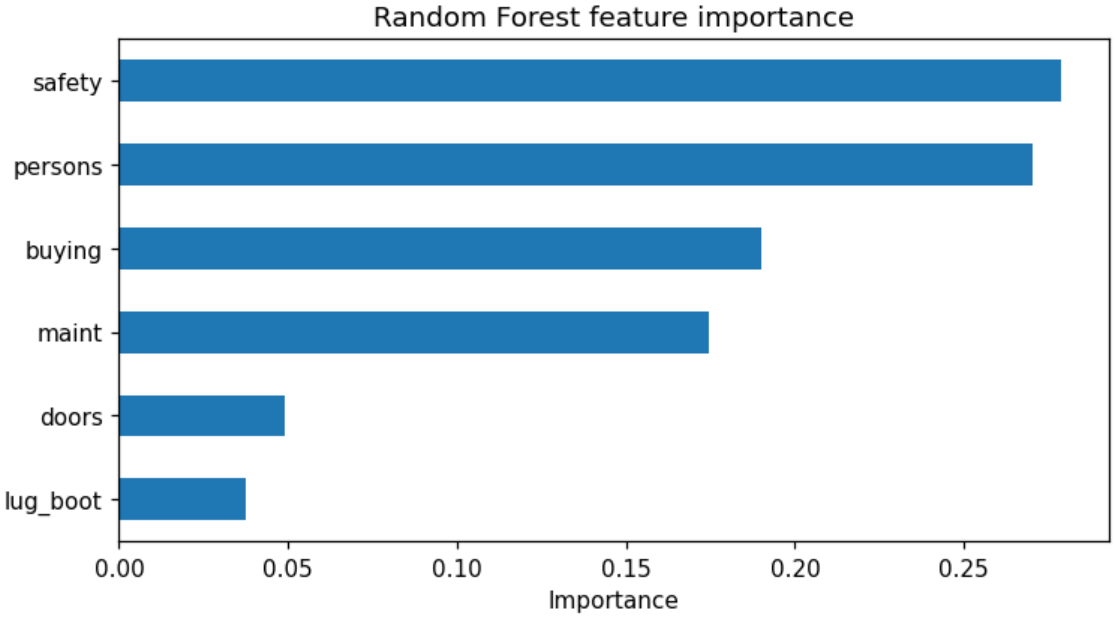
Classification report:
              precision    recall  f1-score   support

         acc       0.97      0.99      0.98       115
        good       0.98      0.93      0.96        59
       unacc       1.00      1.00      1.00       332
       vgood       0.93      1.00      0.96        13

    accuracy                           0.99       519
   macro avg       0.97      0.98      0.98       519
weighted avg       0.99      0.99      0.99       519

Feature importance:
 safety      0.2789
persons     0.2704
buying      0.1902
maint       0.1744
doors       0.0488
lug_boot    0.0373
dtype: float64
```

```
New car: ['low', 'low', '4', '4', 'big', 'high']
Predicted class: vgood
```



Experiment 6: Reinforcement Learning - Tic-Tac-Toe (Q-learning)

Source Code

```
# Experiment 6: Tic-Tac-Toe with Reinforcement Learning (tabular
    Q-learning)
import random
from collections import defaultdict
import matplotlib.pyplot as plt

random.seed(0)

# a/b. Environment: the Tic-Tac-Toe game
    ----------------------------------------
class TicTacToe:
    LINES = [(0,1,2), (3,4,5), (6,7,8), (0,3,6), (1,4,7), (2,5,8), (0,4,8),
         (2,4,6)]
    def __init__(self):
        self.reset()
    def reset(self):
        self.board = [" "] * 9
        return self.state()
    def state(self):
        return "".join(self.board)
    def valid_actions(self):
        return [i for i, c in enumerate(self.board) if c == " "]
    def winner(self):
        for a, b, c in self.LINES:
            if self.board[a] != " " and self.board[a] == self.board[b] ==
                self.board[c]:
                return self.board[a]
        return "draw" if " " not in self.board else None
    def move(self, pos, player):
        self.board[pos] = player
    def show(self):
        b = self.board
        for r in range(3):
            print(" " + " | ".join(b[3*r:3*r+3]))
            if r < 2: print("---+---+---")

# c. Reinforcement learning agent (Q-learning)
    -------------------------------------
class QAgent:
    def __init__(self, alpha=0.3, gamma=0.95, epsilon=1.0):
        self.Q = defaultdict(float)
        self.alpha, self.gamma, self.epsilon = alpha, gamma, epsilon
    def best(self, s, actions):
        qs = [self.Q[(s, a)] for a in actions]
        m = max(qs)
        return random.choice([a for a, q in zip(actions, qs) if q == m])
    def act(self, s, actions, explore=True):
        if explore and random.random() < self.epsilon:
            return random.choice(actions)
        return self.best(s, actions)
```

```
    def update(self, s, a, r, s2, actions2):
        future = max((self.Q[(s2, a2)] for a2 in actions2), default=0.0)
        self.Q[(s, a)] += self.alpha * (r + self.gamma * future -
            self.Q[(s, a)])

REWARD = {"X": 1.0, "O": -1.0, "draw": 0.5}      # agent plays X, opponent
    plays O

def play_episode(env, agent, learn=True, opponent_first=False):
    env.reset()
    if opponent_first:
        env.move(random.choice(env.valid_actions()), "O")
    while True:
        s = env.state(); acts = env.valid_actions()
        a = agent.act(s, acts, explore=learn)
        env.move(a, "X")
        res = env.winner()
        if res is None:                                  # opponent (random)
            replies
            env.move(random.choice(env.valid_actions()), "O")
            res = env.winner()
        if res is not None:
            if learn: agent.update(s, a, REWARD[res], env.state(), [])
            return res
        if learn: agent.update(s, a, 0.0, env.state(), env.valid_actions())

def evaluate(agent, n=1000):
    env, out = TicTacToe(), {"X": 0, "O": 0, "draw": 0}
    for g in range(n):
        out[play_episode(env, agent, learn=False, opponent_first=(g % 2 ==
            1))] += 1
    return out

# d. Train
    ------------------------------------------------------------------------
    ----
agent, env = QAgent(), TicTacToe()
print("Before training (untrained agent vs random opponent, 1000 games):")
print("  ", evaluate(QAgent(epsilon=1.0), 1000), "(X = agent, O =
    opponent)")

EPISODES, hist = 30000, []
for ep in range(1, EPISODES + 1):
    agent.epsilon = max(0.05, 1.0 - ep / (0.6 * EPISODES))
    res = play_episode(env, agent, learn=True,
        opponent_first=(random.random() < 0.5))
    hist.append(res == "X")
    if ep % 5000 == 0:
        print(f"Episode {ep:6d} | epsilon {agent.epsilon:.2f} | recent win
            rate "
              f"{sum(hist[-1000:]) / 1000:.1%} | states learned {len({s for
                  s, _ in agent.Q})}")

# e. Test ----------------------------------------------------------------
```

```
    -----------
agent.epsilon = 0.0
r = evaluate(agent, 1000)
print("\nAfter training (greedy agent vs random opponent, 1000 games):")
print(f"   Wins {r['X']}  Draws {r['draw']}  Losses {r['O']}   -> win rate
    {r['X']/10:.1f} %")

print("\nDemo game (agent = X moves first, random opponent = O):")
random.seed(7)
env.reset()
while env.winner() is None:
    env.move(agent.act(env.state(), env.valid_actions(), explore=False),
        "X")
    if env.winner() is None:
        env.move(random.choice(env.valid_actions()), "O")
env.show()
w = env.winner()
print("Result:", "Agent (X) wins!" if w == "X" else "Opponent (O) wins" if
    w == "O" else "Draw")

# learning curve
k = 500
curve = [sum(hist[i:i + k]) / k for i in range(0, len(hist) - k + 1, k)]
plt.figure(figsize=(7, 4.5))
plt.plot(range(k, len(hist) + 1, k), curve)
plt.xlabel("Training episode"); plt.ylabel("Win rate (per 500 episodes)")
plt.title("Q-learning agent: learning curve"); plt.grid(alpha=0.3);
    plt.show()
```

Experiment 6: Reinforcement Learning - Tic-Tac-Toe (Q-learning)

Output

```
Before training (untrained agent vs random opponent, 1000 games):
   {'X': 406, 'O': 452, 'draw': 142} (X = agent, O = opponent)
Episode   5000 | epsilon 0.72 | recent win rate 51.5% | states learned 3892
Episode  10000 | epsilon 0.44 | recent win rate 65.7% | states learned 4314
Episode  15000 | epsilon 0.17 | recent win rate 81.4% | states learned 4389
Episode  20000 | epsilon 0.05 | recent win rate 90.6% | states learned 4393
Episode  25000 | epsilon 0.05 | recent win rate 89.6% | states learned 4396
Episode  30000 | epsilon 0.05 | recent win rate 88.8% | states learned 4402

After training (greedy agent vs random opponent, 1000 games):
   Wins 923  Draws 76  Losses 1   -> win rate 92.3 %

Demo game (agent = X moves first, random opponent = O):
 X |   | O
---+---+---
 X |   |
---+---+---
 X |   | O
Result: Agent (X) wins!
```

